

Unit -4

Agile Teams, Size and Schedule

4.1 Dynamic System Development Method

4.2 Value-Driven Development

4.3 Team and roles of an Agile Team

4.3.1 Scrum Master

4.3.2 Product Owner

4.3.3 Development Team

4.4 Product Vision and Product Roadmap

4.5 Project Objective and Key Metrics

4.6 Introduction to User Stories

4.7 Estimate the Product Backlog

4.8 Techniques for estimating Story Points

4.9 Plan Product Releases

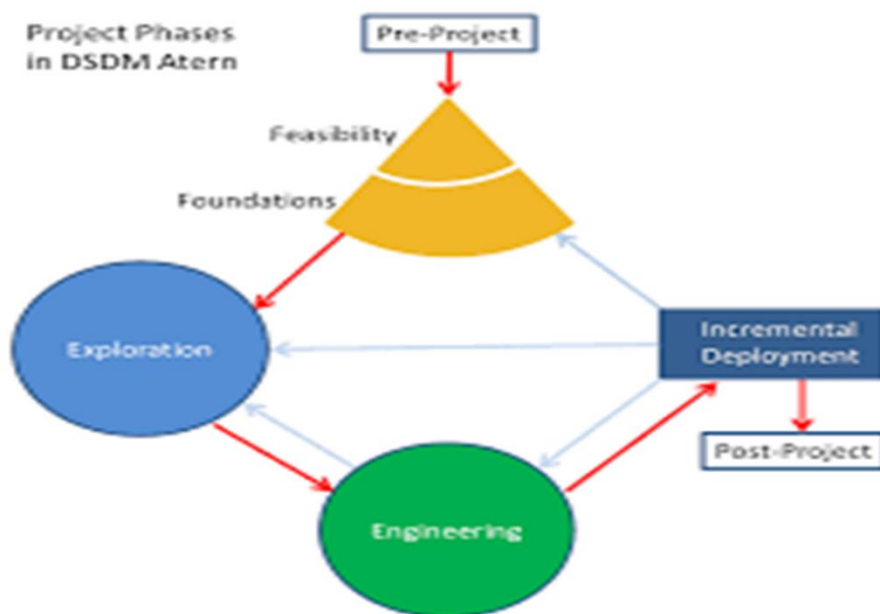
4.10 Product Prioritization

CO3	Demonstrate Agile Teams and Tools and Apply agile project constraints and trade-offs for estimating project size and schedule (Apply)
------------	---

4.1 Dynamic System Development Method

- DSDM is an agile software development methodology.
- It is an iterative, incremental approach that is largely based on the Rapid Application Development (RAD) methodology. The method provides a four-phase framework consisting of: Feasibility and business study.
- The Dynamic Systems Development technique (DSDM) is an associate degree agile code development approach that provides a framework for building and maintaining systems.
- The DSDM philosophy is borrowed from a modified version of the sociologist principle—80 % of an application is often delivered in twenty percent of the time it'd desire deliver the entire (100 percent) application.

- DSDM is an iterative code method within which every iteration follows the 80% rule that simply enough work is needed for every increment to facilitate movement to the following increment.
- The remaining detail is often completed later once a lot of business necessities are noted or changes are requested and accommodated.
- The DSDM tool could be a worldwide cluster of member companies that put together tackle the role of “keeper” of the strategy. The pool has outlined AN Agile Development Model, known as the DSDM life cycle that defines 3 different unvarying cycles, preceded by 2



further life cycle activities:

- **Feasibility Study:**
It establishes the essential business necessities and constraints related to the applying to be designed then assesses whether or not the application could be a viable candidate for the DSDM method.
- **Business Study:**
It establishes the use and knowledge necessities that may permit the applying to supply business value; additionally, it is the essential application design and identifies the maintainability necessities for the applying.
- **Functional Model Iteration:**
It produces a collection of progressive prototypes that demonstrate practicality for the client.

The intent throughout this unvarying cycle is to collect further necessities by eliciting feedback from users as they exercise the paradigm.

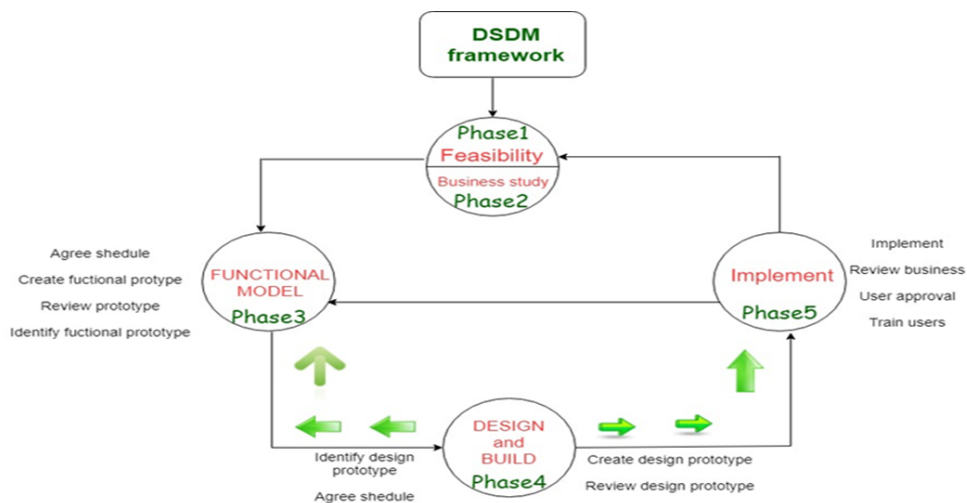
➤ **Design and Build Iteration:**

It revisits prototypes designed throughout useful model iteration to make sure that everyone has been designed during a manner that may alter it to supply operational business price for finish users. In some cases, useful model iteration and style and build iteration occur at the same time.

➤ **Implementation:**

It places the newest code increment (an “operationalized” prototype) into the operational surroundings. It ought to be noted that:

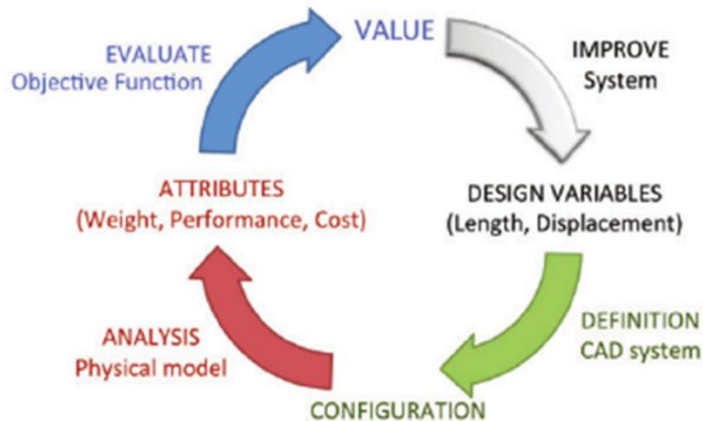
- (a) the increment might not 100% complete or,
- (b) Changes are also requested because the increment is placed into place. In either case, DSDM development work continues by returning to the useful model iteration activity.



Dynamic Systems Development Method life cycle

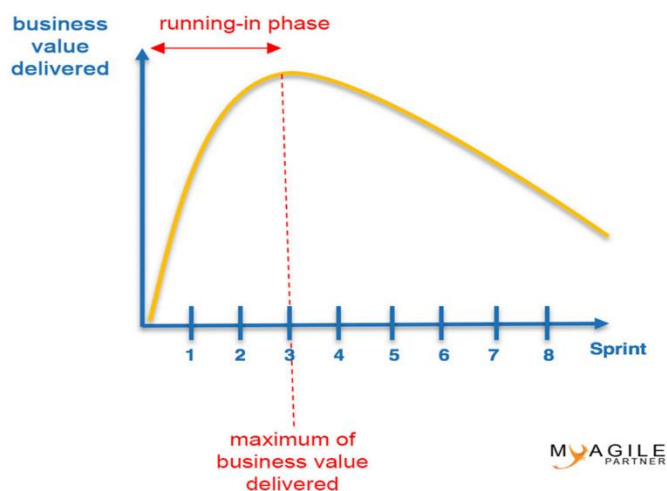
DSDM is often combined with XP to supply a mixed approach that defines a solid method model (the DSDM life cycle) with the barmy and bolt practices (XP) that are needed to create code increments. Additionally, the ASD ideas of collaboration and self-organizing groups are often tailored to a combined method model.

4.2 Value-Driven Development



The **agile manifesto** remember us: “Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.”. And in Value Driven-Development, we try to deliver a maximum of values as soon as possible.

The team will try to follow this curve where it deliver a maximum of values as soon as possible. The team will live a running-in phase at the beginning to setup environment, to create interaction between people, to create the framework...



How to prioritize in Value Driven-Development?

The team is agile and it works by small lot (user-stories). These user-stories have to be INVEST because the Value Driven-Development (VDD) works only with independent items.

Determine the business value

Each user-story will have a business value determined by the product owner with the help of users (or key users). We will use the scale 100 to 1000 to give a business value at each user stories.

***For example:** user-story 1 = 900 BV – user-story 2 = 200 BV*

Determine the effort of development

But it's not sufficient to deliver a maximum of values as soon as possible with business value because the effort of development will have a real impact on the prioritization.

So, we will determine the story points of each user story in the Value Driven-Development concept. For that, we will use the fibonacci numbers: 1, 2, 3, 5, 8, 13.

***For example:** user-story 1 = 13 SP – user-story 2 = 2 SP*

What is a story point?

The story point of a user story (or a technical task) will include different notions that developers will have to take into account to estimate it:

- the effort to develop the request
- the difficulty (complexity) of the request
- the risks we imagine to meet during the development of the request
- any unknowns existing at the time of the estimate
- potential dependencies with external elements

We must accept that the point of effort is an “abstract” notion that can't be compared to a number of man days. But it allows to make predictability.

Yes, the point of effort takes a notion of time contrary to what we can read sometimes. But his estimate isn't just based on it and this notion of time is not materialized by 1 story point = 1 day.

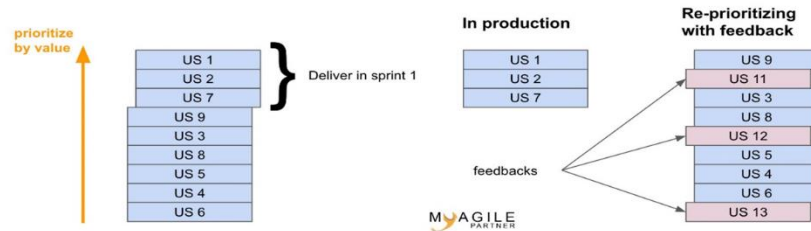
Calculate the ROI (return on investment)

When we have the business value and the story point for each user-story, we can calculate the ROI (return on investment). It's this indicator that help us to prioritize in doing Value Driven-Development (VDD).

Here is the method to calculate the ROI

ROI= BV/SP

Now, you have the best element to prioritize by the value your backlog. So, the product owner will range the user-stories by ROI



Value Driven-Development (VDD) to deliver by value – Value Driven-Development

The product owner will prioritize again the backlog with the new user-stories created from user feedbacks; sometimes the feedbacks will have a very important ROI for the product and could change the prioritization of the backlog.

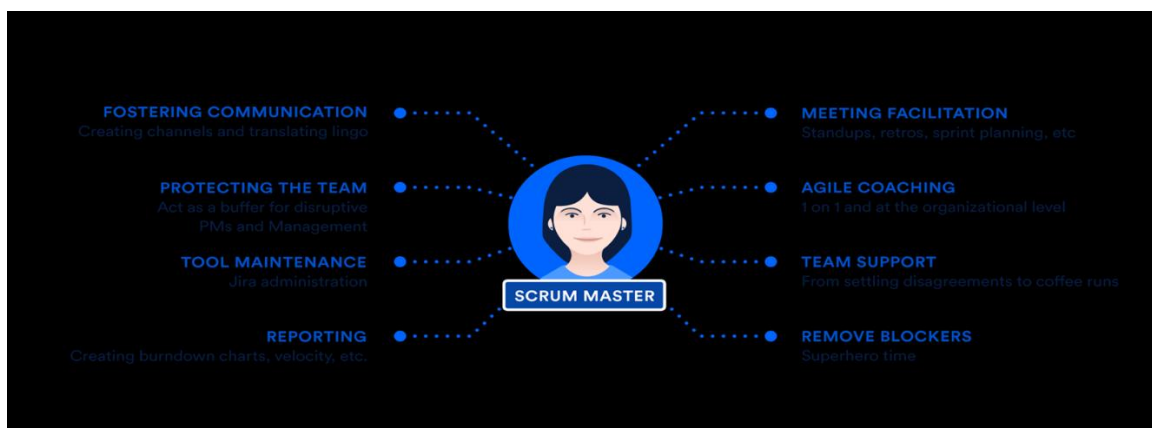
4.3 Team and roles of an Agile Team

4.3.1 Scrum Master

What is a scrum master?

A Scrum master is the facilitator of scrum, a lightweight agile framework focusing on time-boxed iterations called sprints. Scrum masters act as coaches to the rest of the team, or *servant leaders*, as the Scrum Guide puts it.

Good scrum masters are committed to the foundational elements of scrum but remain flexible and open to opportunities for the team to improve their workflows.



Scrum master responsibilities

1. **Standups:** Facilitate daily standups (or the daily scrum) as needed.
2. **Iteration/sprint planning meetings:** Protect the team from over-committing and scope creep. Aid in estimation and sub task creation.
3. **Sprint reviews:** Participate in the meeting and capture feedback.
4. **Retrospectives:** Note areas for improvement and action items for future sprints.
5. **Board administration:** Work as the administrator of the scrum board. Ensure that cards are up to date and the scrum tools like Jira software are working well.
6. **1 on 1s:** Meet individually with team members and stakeholders as needed. Iron out team disagreements about process and work styles. Many scrum practitioners oppose 1on1, believing these communications should happen during standups. However, new teams often prefer to have these regular face-to-face interactions with specific team members. The scrum master may decide that these individual interactions are crucial for team development and getting to know one another.
7. **Internal Consulting:** Scrum masters should consult with team members and internal stakeholders on how best to work with the scrum team.
8. **Reporting:** Regular analysis of burndown charts and other portfolio planning tools to understand what gets built and at what cadence.

Scrum master skills

1. **Leadership:** Scrum masters must exhibit leadership qualities without being authoritative. They lead by serving the team and facilitating their self-organization.
2. **Effective communication:** Clear and transparent communication is crucial. Scrum masters ensure all team members and stakeholders understand the project's status, goals, and obstacles.
3. **Empathy:** It is essential to understand and empathize with the team's challenges. Scrum masters need to support and motivate team members during difficulties.
4. **Problem-solving:** Scrum masters often deal with impediments and conflicts. They must be adept at problem-solving and finding solutions to keep the team on track.
5. **Adaptability:** Scrum masters should be adaptable and open to change. They need to adjust to the evolving needs of the team and the project.
6. **Facilitation skills:** The ability to effectively facilitate meetings and scrum ceremonies is vital. Scrum masters ensure these events run smoothly and focus on the intended outcomes.

7. **Coaching and mentoring:** Scrum masters coach and mentor team members in agile and scrum principles, helping them continually improve.
8. **Technical knowledge:** While not mandatory, having a technical background can be beneficial, especially when working with development teams. It allows scrum masters to better understand the technical challenges faced by the team.
9. **10. Time management:** Scrum masters must manage time effectively to keep the team on schedule and meet sprint goals.

Scrum team

The scrum team is a fundamental aspect of the scrum framework. It typically consists of the following roles:

Development team: These are professionals with the skills required to deliver a potentially shippable product increment. The development team is responsible for the actual work of creating the product.

Product owner: The product owner represents the interests of the stakeholders and is responsible for managing the product backlog and prioritizing work.

Scrum master: As discussed in the previous sections, the scrum master is pivotal in facilitating the scrum process and ensuring that the team adheres to scrum principles.

Scrum framework

The scrum framework is a structured approach to agile project management methodology. It consists of several components, including:

1. **Sprints:** Sprints are time-boxed iterations, typically two-to-four weeks in length, where the development team works to complete a set of planned work items.
2. **Product backlog:** The product backlog is a prioritized list of all the features, enhancements, and bug fixes the product needs to address. The product owner manages and maintains this backlog.
3. **Sprint backlog:** The sprint backlog is a subset of the product backlog, containing the work items selected for a specific sprint.
4. **Scrum artifacts:** Scrum employs various artifacts, such as burndown charts and task boards, to track progress and make the work visible to the team and stakeholders.

5. **Scrum events:** Scrum events, including sprint planning, daily stand-ups, sprint reviews, and sprint retrospectives, provide structure and opportunities for collaboration within the team.

When teams need a scrum master

Any scrum trainer will teach that a scrum team must have a scrum master. Without one, you are doing something shy of a true scrum, often called scrum-but.

When starting with scrum, it can be a huge help to have someone in the role who has seen scrum work before. For this reason, scrum masters are often hired as consultants or fractional employees rather than as full-time staff members. However, every scrum team is different.

Many experienced teams handle the responsibilities listed above as a unit and take pride in sharing management duties. The scrum master role may rotate throughout the team, with different team members taking turns facilitating standups and retrospective meetings. Other teams may have the same person play the scrum master role every day.

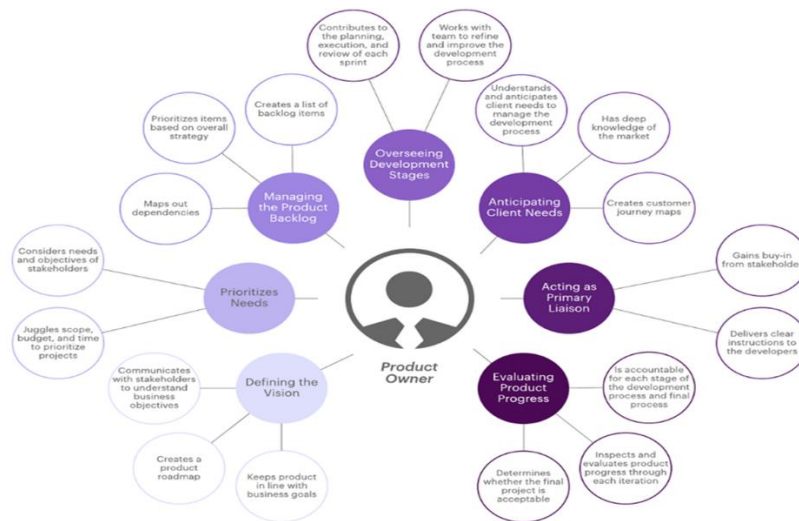
Unfortunately, misunderstanding the scrum master role often leads existing managers to assume it is their role. To better understand why this can be a problem, let's compare the scrum master job description to non-scrum roles you may already have in your company and examine why keeping the roles separate is important.

4.3.2 Product Owner

A product owner is responsible for ensuring the success of a project in Scrum. The product owner is responsible for managing and optimizing the product backlog in order to maximize the value of the product. A Scrum framework is an Agile methodology that facilitates communication and self-organization within a team.

A Product Owner is part of the scrum team. The key responsibilities of a Product Owner are to define user stories and create a product backlog. The Product Owner is the primary point of contact on behalf of the customer to identify the product requirements for the development team. This product backlog will be a prioritized set of customer requirements. The Product Owner has the complete responsibility and ownership of defining and even prioritizing user requirements. The Product Owner must communicate with the development team to explain the product features to be implemented. Any queries that come from the development team must be addressed by the Product Owner on key user requirements. The role of the Product Owner is to maximize the value addition of the products that are developed by the agile scrum team.

The Product Owner must ensure that the user stories meet customer requirements. The role of the Product Owner is critical for companies that are keen to move to an agile-based product development methodology. The Product Owner has to collaborate and work closely with various stakeholders such as customers, business leaders, development teams, project managers, and other stakeholders.



What Does a Product Owner Do?

- Defining and managing the product vision and strategy, based on customer and stakeholder needs and market research
- Creating and prioritizing a product backlog (a list of features and requirements) that aligns with the product vision and goals, and continuously refining it based on feedback and changing business needs.
- Collaborating closely with cross-functional teams (e.g. developers, designers, marketers, and quality assurance) to ensure that the product meets customer needs and is delivered on time and within budget.
- Ensuring that the product backlog items are clearly defined, well understood, and properly estimated by the development team.
- Making tough decisions on what features to include in each sprint or release, based on the value they will deliver to customers and the business.
- Acting as the primary point of contact for all stakeholders (e.g. customers, partners, executives, and other departments) on matters related to the product.

- Continuously monitoring the product's performance and gathering feedback from customers and stakeholders, using data-driven insights to make informed decisions and prioritize future improvements.
- Maintaining a deep understanding of the competitive landscape, emerging trends, and new technologies that may impact the product's success.
- Ensuring that the product is compliant with all relevant regulations and standards, and managing any legal or ethical considerations related to the product.
- Demonstrating leadership and ownership of the product, and inspiring others to share and contribute to the product vision and goals.

Product Owner Skills

Product Management

The product Owner should be able to define, prioritize, and manage product features and requirements, identify market opportunities and understand customer needs.

Agile Development

A Product Owner should be familiar with Agile development methodologies [Scrum, Kanban or Lean]. Understanding these methodologies and frameworks will help you manage your product backlog, plan and conduct sprint reviews, and work effectively with your development team.

Communication and Collaboration

Strong communication skills are essential for any Product Owner. You must effectively communicate the priorities, product vision, and requirements to the development team, stakeholders, and customers. Additionally, you should be able to collaborate effectively with cross-functional teams, including developers, designers, marketers, and salespeople.

Business Analysis

Experience in business analysis, including identifying customer needs, analyzing data, and developing business cases, will help Product Owners make data-driven decisions about your product and ensure it aligns with your business objectives.

Development Team

A **development team** is a group of people who work together to develop a piece of software, product, or service from initial ideation to completion. While many people use the term as short-hand to refer to a software development team (which develops

software), a project development team can actually be any team focused on developing a particular project, whether it be constructing a building or manufacturing a new toy.

A project development team often includes a project manager, who oversees the planning and execution of the project, as well as project support members like a project coordinator. The roles of additional team members will depend on the nature of the project but can include both internal and external stakeholders who contribute to the project's development in different ways. There are many ways to organize a development team. While some are organized in strict hierarchies in which everyone's roles and responsibilities are clearly defined, many others consist of small, cross-functional teams in which all members contribute to the development and completion of the entire project (an Agile methodology known as "Scrum").

Ultimately, much as the term "development team" can refer to many different *types* of teams focused on different goals, the structure of the team will differ based on their overall objectives and the philosophy of their employer.

Role and Responsibilities in the Agile Software Development Team

- **Business Analyst (BA)**

This is someone who is responsible for formulating goals, analyzing and documenting the core processes and systems, and ensuring the alignment of business model and technology. BA is all things for everybody. They evaluate what works and what doesn't work and set the direction for business development.

- **Project Manager (PM)**

This person is in charge of planning and execution. PM is responsible for getting things done. They also take care of building relationships among the client and various organization departments. PMs oversee all the processes, delegate the tasks among other team members, and ensure that everyone stays on track.

- **UX Designer**

This is someone who designs the way users will interact with the product. They ensure that all the features solve people's problems and fulfill business goals. Namely, they determine how the product will look and how it will work. The main focus of a UX designer is functionality and usability.

- **Developers (Front-end/ Back-end)**

These are the people who do the actual coding. Whereas front-end developers work on the customer-facing elements of the product, back-end developers take care of its functionality, which is everything the user doesn't see.

- **Quality Assurance Engineer (QA)**

This is someone who tests the product to make sure that it works well, meets the quality standards and client requirements. QA is like a final editor with meticulous attention to the smallest detail. They detect errors and bugs early on so that the team can fix it before it gets to the users.

Traditional team vs agile team

Traditional team	Agile team
Top-down project management. The project manager is responsible for getting things done.	Self-organized and self-managed team. PM's role is to coach the team, remove obstacles, and prevent distractions.
Teams may work on several projects simultaneously.	Teams focus on one project at a time.
An organization evaluates individual performance	An organization evaluates team performance
Distinct roles and job titles.	Cross-functional teams, skills matter more than titles.
No team size limits.	Three to nine people per team.
Employees are referred to as <i>human resources</i> .	Employees are referred to as <i>talents</i> .

Product Vision

- The Product Vision describes the purpose of a Product. A good Product Vision expresses the value the Product should deliver and to whom that value is delivered.
- It is effective when people connect with the vision emotionally and practically.

- When a Product Vision is both aspirational and actionable, it inspires creativity within the Scrum Team allowing them to collaboratively work with Stakeholders on how they might work toward the vision.
- The Product Vision is owned by the Product Owner, but its development requires input from stakeholders and the Scrum Team(s).
- A good Product Vision is built incrementally allowing many opportunities for inspection and adaptation.
- The Product Vision should be a free-standing, transparent instrument that enables stakeholders and the Scrum Team to understand the intention of a Product. However, even though it should stand on its own, a Product Vision exists in the context of two other critical elements: Business Strategy and Product Strategy.

Key aspects of Product Vision:

1. **Clarity of Purpose:** A good product vision clearly articulates why the product exists and the problem it aims to solve. It helps everyone involved understand the product's core purpose and the value it brings to users.
2. **Inspiration:** The vision should inspire and motivate the team.
3. **Alignment:** The vision ensures that all team members, stakeholders and users are aligned towards a common objective. This alignment is crucial for making consistent decisions throughout the development process.
4. **Adaptability:** while the vision provides a long-term direction, it should also allow for adaption. Agile methodologies emphasize responding to change, so the product vision needs to be flexible enough to accommodate evolving requirements and market conditions.
5. **Communication :** Effectively communicating the product vision is key. It should be concise and easily understandable by all stakeholders . This helps in avoiding misunderstanding and ensures that everyone is working towards the same goal.
6. **Measurable Outcomes:** While the vision is often qualitative, its beneficial to tie it to measurable outcomes. This allows the team to gauge progress and success, aligning with agile princi[les of delivering value and continuously improving.



Product Strategy :

The Product Vision tells us where we are going and the Product Strategy outlines on a high level how we believe we will be getting there.

If the Product Vision and Product Strategy are not shared they will not be effective. The point is that everyone should know where we are going and how we intend to get there so everyone can actively participate to their full capacity.

Key components you need to thoroughly understand to build a solid Product Strategy are:

1. Your customers and users
2. Your company and your product
3. Sales and marketing
4. Competitors
5. Market trends
6. Risks

The Product Strategy is normally closely aligned with the Business Strategy of the company. In a company with only one product they can be more or less the same. In some companies with multiple products there might be freedom to experiment with new business ideas that are not tightly connected to the core business strategy.

product roadmap

- *A product roadmap is a plan of action for how a product or solution will evolve over time.*
- *Product owners use roadmaps to outline future product functionality and when new features will be released.*

- *When used in agile development, a roadmap provides crucial context for the team's everyday work and should be responsive to shifts in the competitive landscape.*
- A product roadmap is essential to communicating how short-term efforts match long-term business goals. Understanding the role of a roadmap—and how to create a great one—is key for keeping everyone on your team headed in the same direction.
- A product roadmap is a shared source of truth that outlines the vision, direction, priorities, and progress of a product over time.
- It's a plan of action that aligns the organization around short and long-term goals for the product or project, and how they will be achieved.
- Product owners use roadmaps to collaborate with their teams and build consensus on how a product will grow and shift over time. Agile teams refer back to the product roadmap to keep everyone on the same page about which product ideas have been prioritized and when, and to gain context for their everyday work and future direction.

Which teams use product roadmaps?

Roadmaps come in several different forms and serve a variety of audiences:

Internal roadmap for the development team: These roadmaps can be created in several ways, depending on how your team likes to work. Some common versions include the detail about the prioritized customer value to be delivered, target release dates and milestones. As a general rule, development teams should use a product roadmap to understand the product strategy, how it connects to goals, and why initiatives have been prioritized. For the actual development work, dev teams should create a separate delivery plan that maps back to the product roadmap. Since many development teams use agile methodologies, these plans are often organized by sprints and show specific pieces of work and problem areas plotted on a timeline.

Internal roadmap for executives: These roadmaps emphasize how teams' work supports high-level company goals and metrics. They are often organized by month or by quarter to show progress over time towards these goals, and generally include less detail about detailed development stories and tasks.

Internal roadmap for sales: These roadmaps focus on new features and customer benefits, and may even include key customers who are interested in these features in order to support sales conversations. An important note: avoid including hard dates in sales roadmaps to avoid tying internal teams to potentially unrealistic dates.

External roadmap: These roadmaps should excite customers about what's coming next. Make sure they are visually appealing and easy to read. They should provide a high-level,

generalized view of new features and prioritized problem areas to get customers interested in the future direction of the product.

Why are product roadmaps important?

The biggest benefit of the product roadmap is the strategic vision it illustrates to all stakeholders. The roadmap ladders up to broader product and company goals with development efforts, which connects work across teams and aligns those teams around common goals to create great products.

- **For organizational leadership,** the roadmap provides updates on the status of planned features and improvements in a format that connects back to company goals and is easily understood.
- **For product owners and managers,** roadmaps unify teams working on high impact product enhancements and allow them to communicate priorities and why they were prioritized effectively with adjacent teams.
- **For the developers themselves,** roadmaps provide a better understanding of the “big picture,” which allows team members to focus on the most important tasks, avoid scope creep, and make fast, autonomous decisions.

	1 st quarter	2 nd quarter	3 rd quarter	4 th quarter
	Version 1	Version 2	Version 3	Version 4
	Acquisition: Free app, limited in-app purchases	Activation: Focus on in-app purchases	Retention	Acquisition: New segment
	• Basic game functionality • Multiplayer • FB integration	• Purchase dance moves • Create new dances	• New characters and floors • Enhanced visual design	• Street dance elements • Dance competition
	Downloads: top 10 dance app	Activations, downloads	Daily active players, session length	Downloads

4.5 Project Objective and Key Metrics



Key Metrics:

Metric: it is used to measure and improve the teams performance . Common metrics include velocity (amount of work completed in a sprint) , Sprint burndown(progress tracking), and lead time (time taken to deliver feature). These metrics help teams assess their efficiency and identify areas for improvement.

KPI : Key Performance Indicators (KPIs) in Agile help teams and organizations measure their success and progress toward strategic goals. Some common Agile KPIs include:

1. **Velocity:** Measures the amount of work a team can complete in a sprint, providing insights into productivity.
2. **Sprint Burndown:**Tracks the completion of tasks within a sprint, helping to visualize progress.
3. **Lead Time:** Measures the time it takes to deliver a feature from ideation to implementation, offering insights into the efficiency of the development process.
4. **Cycle Time:**Similar to lead time, it focuses on the time it takes to complete a task once work has started.
5. **Burndown and Burnup Charts:** Visual representations of completed work versus planned work over time.
6. **Defect Density:** Measures the number of defects per unit of code, indicating code quality.
7. **Customer Satisfaction:**Gauges the satisfaction of end-users or customers with the delivered product increments.

Selecting the right KPIs depends on project goals and the specific aspects of performance that need monitoring and improvement.

OKR: Objectives and Key Results (OKR) is a goal-setting framework often used in Agile environments. Here's a breakdown:

1. **Objectives (O):** These are high-level, qualitative goals that **define what needs to be achieved**. They set a clear direction for the team.

2. **Key Results (KR):** These are specific, measurable outcomes that indicate **whether the objectives have been achieved**. Key Results are quantifiable and provide a way to track progress.

In an Agile context:

Alignment: OKRs help align teams and individuals with organizational goals. Everyone understands what needs to be achieved.

Flexibility: OKRs allow for adaptability in a rapidly changing Agile environment. Teams can adjust their objectives and key results as priorities shift.

Transparency: The framework promotes transparency as everyone can see the objectives and key results, fostering a shared understanding of priorities.

Implementing OKRs in Agile promotes a results-oriented and collaborative approach, fostering continuous improvement and adaptation to changing circumstances.

Project Objectives:

In Agile, project objectives are often framed as high-level goals that guide the team's work. These objectives help provide a clear direction and purpose for the project. Some characteristics of project objectives in Agile include:

1. **User Value Focus:** Objectives typically emphasize delivering value to end-users. Agile prioritizes customer satisfaction and the continuous delivery of valuable increments.

2. **Measurable Outcomes:** Objectives are more effective when they are specific and measurable. This allows teams to assess their progress and success against concrete criteria.

3. **Adaptability:** Agile objectives should be adaptable to change. The iterative nature of Agile allows for adjustments based on feedback, emerging priorities, or shifts in the business environment.

4. Alignment with Business Goals:Project objectives should align with broader organizational goals. This ensures that the project contributes to the overall success and strategy of the business.

5. Time-Bound: Agile projects often work in time-boxed iterations or sprints. Objectives may have time-bound elements to ensure timely delivery and continuous improvement.

6. Collaborative Definition:Agile encourages collaboration, and the process of defining project objectives is often a collaborative effort involving stakeholders, product owners, and team members.

4.6 Introduction to User Stories

A user story or agile/ scrum user story is a tool that's used in agile software development and product management to represent the smallest unit of work in the framework. It provides an informal, natural language description of a feature of the software or product from the end-user perspective.

A good way to think about a user story is that it is a reminder to have a conversation with your customer (in XP, project stakeholders are called customers), which is another way to say it's a reminder to do some just-in-time analysis. In short, user stories are very slim and high-level requirements artifacts

Initial User Stories (Informal)

user stories are small, much smaller than other usage requirement artifacts such as use cases or usage scenarios.It's important to recognize that each of the statements in represents a single user story.

Example:

- Students can purchase monthly parking passes online.
- Parking passes can be paid via credit cards.
- Parking passes can be paid via PayPal.
- Professors can input student marks.
- Students can obtain their current seminar schedule.
- Students can order official transcripts.
- Students can only enroll in seminars for which they have prerequisites.

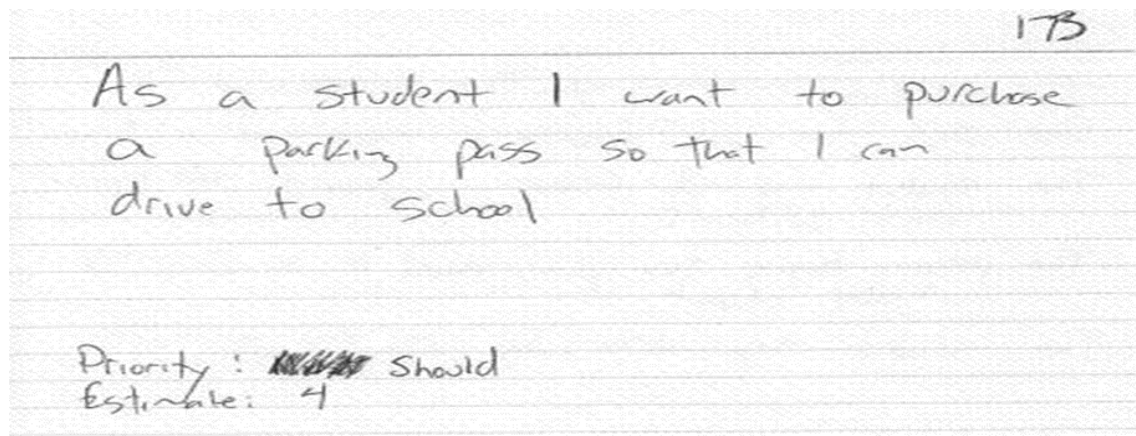
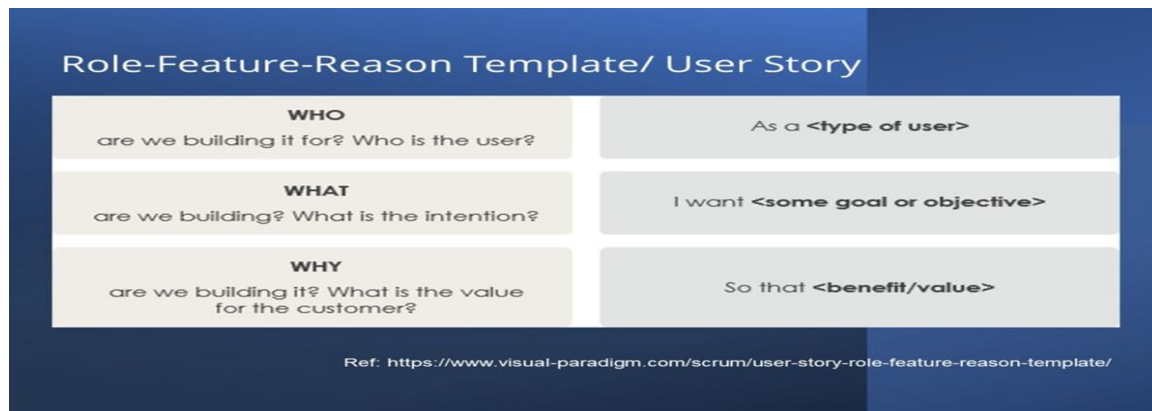
- **Transcripts will be available online via a standard browser.**

Important considerations for writing user stories:

- **Stakeholders write user stories.** An important concept is that your project stakeholders write the user stories, not the developers. User stories are simple enough that people can learn to write them in a few minutes, so it makes sense that the domain experts (the stakeholders) write them.
- **Use the simplest tool.** User stories are often written on index cards as you see in Figure 2 (at least when your project team is co-located). Index cards are very easy to work with and are therefore an inclusive modeling technique.
- **Remember non-functional requirements.** Stories can be used to describe a wide variety of requirements types. For example in Figure 1 the *Students can purchase parking passes online* user story is a usage requirement similar to a use case whereas the *Transcripts will be available online via a standard browser* is closer to a technical requirement.
- **Indicate the estimated size.** You can see in Figure 2 that it includes an estimate for the effort to implement the user story. One way to estimate is to assign user story points to each card, a relative indication of how long it will take a pair of programmers to implement the story. The team then knows that if it currently takes them on average 2.5 hours per point; therefore the user story in Figure 2 will take around 10 hours to implement.
- **Indicate the priority.** Requirements, including defects identified as part of your independent parallel testing activities or by your operations and support efforts, are prioritized by your project stakeholders (or representatives thereof such as product owners) and added to the stack in the appropriate place. You can easily maintain a stack of prioritized requirements by moving the cards around in the stack as appropriate. You can see that the user story card includes an indication of the priority; I often use a scale of one to ten with one being the highest priority

Initial User Stories (Formal)

As a (role) I want (something) so that (benefit).

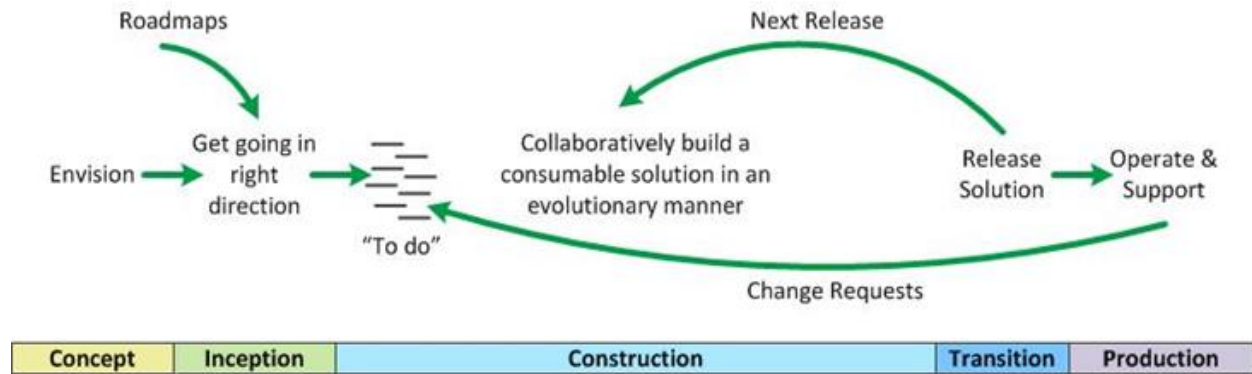


User Stories throughout the Agile Life Cycle

Inception. You often create a stack of user stories during Inception as part of your requirements envisioning activities to identify the scope of your system.

Construction. During construction iterations you will identify new stories, split existing stories when you realize that they're too large to be implemented in single iteration, reprioritize existing stories, or remove stories that are no longer considered to be in scope. The point is that your stories evolve over time just like other types of requirements models evolve. Furthermore, enhancement requests may be identified by your support staff during the production phase and then forwarded to a development team as they are working on an upcoming release. These enhancement requests are effectively new stories (albeit in a different format).

Transition. Sometimes new stories will be identified during the Transition phase, although this isn't very common as the focus of release is on hardening the system and not on new functionality. But it does happen, and these stories would be prioritized and placed on the stack in priority order just as you normally would.



Epics

An agile epic is a body of work that can be broken down into specific tasks (called user stories) based on the needs/requests of customers or end-users.

Themes:-

A theme is a collection of features or epics that are grouped together based on a common goal or area of functionality. Themes help to organize and prioritize the work that needs to be done and align it with the overall goals of the project.

What makes a good user story?

Agile teams assess the quality of stories by using the [INVEST acronym](#). This stands for:

- **Independent:** The user story should be independent of all others. Because they are not connected, they can be worked on in any order.
- **Negotiable:** A user story should be flexible enough to allow for negotiation between the customer and product owner.
- **Valuable:** What value does the user story bring? If you cannot find any value, the story should not be completed.
- **Estimable:** You should be able to estimate how long a user story will take so that you can effectively manage your time.
- **Small:** The story must be small enough to be completed within a single sprint.
- **Testable:** You must be able to test your user story in line with quality assurance standards.

If a user story does not meet the INVEST criteria, it should be rewritten or removed from the epic. However, if it does, your team members can get to work. Schedule daily [Agile meetings](#) to check on their progress and ensure they are on track to complete the user story within the [sprint timeframe](#).

Benefits of user stories

- **Simplified format:** User stories are written in easy-to-understand language. This eliminates confusion and makes it easier to grasp what the customer is looking for.
- **Increased flexibility:** Because user stories don't go into technical detail, they can be molded to fit changing situations.
- **Improved collaboration:** When team members are aligned on one goal, they can work better together and collaborate easily with other project stakeholders

Though the benefits of writing user stories are significant, a product manager must also consider the potential disadvantages.

Example of User Story:

As a Banking Customer, I want to access Savings account so that I can view/make transactions

As a Banking Customer, I want to access Credit Card page, so that I can view and make transactions

As a Banking Customer, I want to access Loans page so that I can view my loans

As a Banking Customer, I want to transfer funds, so that I can move my funds to different accounts within my bank and other banks

As a Banking Customer, I want to access/view summary of my savings account, so that I know my balance and other details

As a Banking Customer, I want to Login to Net banking so that I can view credit card details

As a Banking Customer, I want to transfer funds within my own accounts so that I can move some balance across my accounts

As a Banking Customer, I want to transfer funds from my account to another account in another bank, so that I can send money to my family and friends who have accounts in other banks

As a Banking Customer, I want to add beneficiary to my account, so that I can transfer funds to the beneficiary

As a Net Banking Administrator, I want to have the customer's data backed up so that I can restore it any time in case of issues

As a Net Banking application, I want to shake hands with another bank using a specific formatted XML so that funds can be transferred based on the customers' needs.

Product Backlog – A Brief Overview

A product backlog is a prioritized list of work that the development team is meant to deliver. It is derived from the roadmap and its requirements and presents the most important items at the top. This way, it ensures that the team is focusing on the most valuable and important features, fixing critical bugs, or doing other crucial tasks.

The most common element of a product backlog is user stories which are general, informal description of a software feature that is written from the end-user perspective. In most cases, not all user stories are equal, as items usually have different importance levels, different sizes, and description levels.

5 Reasons Why Product Backlog Estimation is Important

A well-prioritized and estimated product backlog not just streamlines iteration and release planning, it also lets the team have a clear picture of all the things on which the members are going to spend time.

When a team sits together and starts estimating product backlog items, they are basically getting all the know-how of the list of prioritized works that they are going to work on from now onwards. So, what makes product backlog estimation essential can be understood from the below 5 reasons:

1. Better Team Discussion and Decision Making

Not all team members can depict the complexity and challenges of product backlog items. Some members are more experienced or they have a better understanding, so they can pinpoint hidden complications associated with the tasks that others fail to detect.

Therefore, one of the reasons to estimate product backlog is to have better team discussion so that all aspects of the backlog items are discussed before the development begins. This way, the team can interact more focused, newbies can learn from seniors, and better decisions can be made effectively.

2. Better Prioritized Tasks

It is the duty of the product owner to design the product backlog and prioritize the items rightly. However, there are always chances that some items might not be set at the right priority level. For example, the product owner might set one feature to be part of the minimum viable product

(MVP), but it could have been delayed till the next release. Similarly, there might be some features/bugs that need urgent attention compared to others.

So, when a team gathers for estimating product backlog items, they will also evaluate the importance of items for the product's success. The individual discussion on the product backlog items and their estimations will make a clear picture of what items should be on the top of the priority list. This way, the product backlog can be set in a more prioritized form that is in line with the team's interest and the product's success.

3. Better Time Management

Estimating a product backlog requires team members to gather together for the estimation session and carry out an estimation of the product backlog items. So, it might give a sense of time wastage, as members are forced to take part in estimation. However, the reality is a bit different.

When a team gathers and carries out product backlog estimation, it is basically setting a mutually-discussed path for project completion. They are prioritizing the tasks, discuss the efforts/time needs, and all other aspects associated with the project. Overall, it increases team productivity and focus, thereby causing a positive impact on time management and timely deliveries.

4. Increase Efficiency

It is a common concept that you remain more focused on any work when you know its outcomes. The same implies to the Scrum team. The efficiency of the team can increase if they know what are the goals they are targeting to achieve.

With product backlog estimation, the team becomes familiar with the main goals behind the project, the complications/challenges they have to deal with, the efforts they have to input, and the estimated time of completion. This way, all the members become familiar with all the minor to major details of the project and are in a better position to carry out iterative development stages more focused.

5. Align Expectations

The product backlog is more like a visual representation of the current development process. When a team estimates product backlog, it is basically creating a mutual understanding of the project and the tasks it is meant to complete.

So, another benefit of product backlog estimation is that it helps in aligning members' expectations via a single source of data, which assists them in better collaboration to achieve common objectives.

Best Product Backlog Estimation Technique

It is not always easy to estimate product backlog because every team member has their own views on the particular item and a mismatch in thoughts can sometimes make a troublesome situation. However, challenges associated with product backlog estimation can be addressed effectively with the right estimation technique.

Planning Poker

Out of many available product backlog estimation techniques Planning Poker shines as the best one. Planning Poker is a relative estimation technique based on the consensus that involves a numeric scale, which is mostly modified Fibonacci sequence (1, 2, 3, 5, 8, 13, 20, 40, and 100). How planning poker works can be very well understood from the below steps:

1. Team members join the product backlog estimation session. They are provided with poker-style cards that contain the numeric value that they will use later for estimating stories.
2. Once all members are present, the product owner presents the story and lets the members discuss to clear their concepts.
3. After discussion, every member gives the numeric value to the story based on what they think best reflects the efforts required to complete it.
4. If all members have picked the same value, then that is declared the team estimate. However, if there is a mismatch, then members that have assigned the highest and lowest values are asked to present their reasons. Afterward, the discussion is carried out until a final team consensus is reached.
5. The team follows the above steps again and again until all the stories (product backlog items) are estimated.

Planning Poker encourages a collaborative approach, which results in accurate estimates and a more committed team. The only concern with Planning Poker estimation is that it requires a team to sit together at once and complete all estimates. So, if some members are unable to attend the session or there are newbies on the team, then the accuracy of estimates is a concern.

If teams face the above challenge, then they can opt for a modified Planning Poker technique, i.e., asynchronous planning poker. Async Poker or Asynchronous Planning Poker technique is best to deal with remote or distributed teams. Following are the steps involved in this technique:

1. The moderator emails the stories to be estimated to all the members and give them the deadline to provide their estimates.
2. Members read and understand the stories at their own pace and then provide their estimates.

3. Once the moderator receives all estimates and the values are close to each other, then those estimates are declared the team's final estimates. However, if there are mismatches, then a quick session is conducted either on-premises or online to clear doubts and reach a consensus.

This way, Asynchronous Planning Poker elevates the whole estimation process, reduces time spent by members, and gives everyone the time to read and understand stories at their own pace.



T-Shirt Sizes

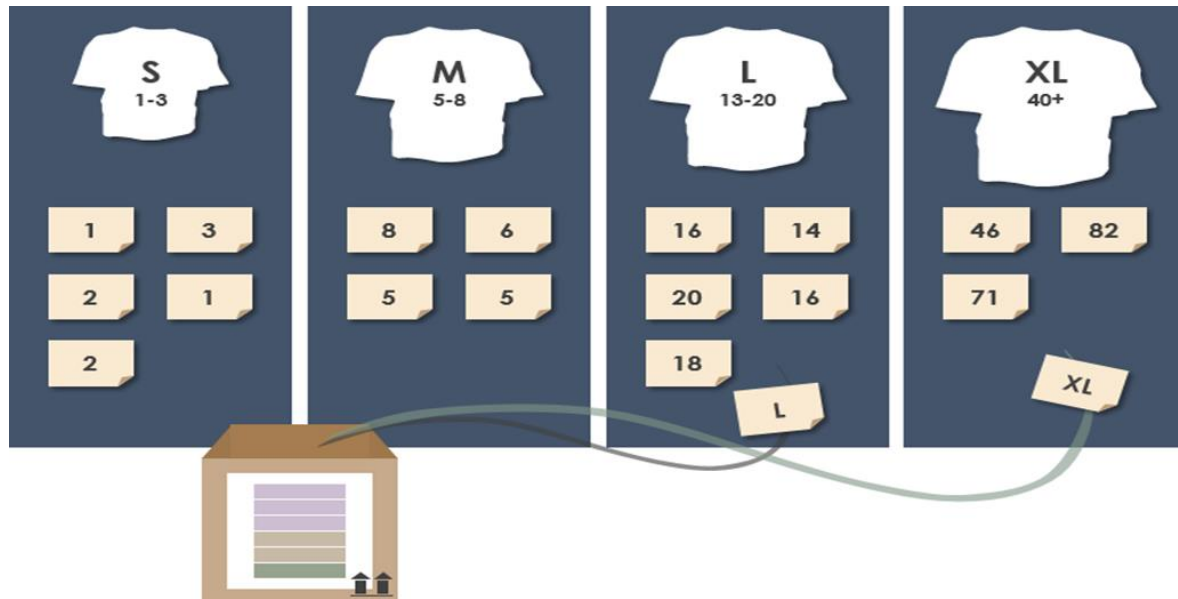
If you think about T-shirts, there are multiple sizes to choose from. More specifically – there is extra-small (XS), small (S), medium (M), large (L) and extra-large (XL). This technique uses these sizes as story points for the size of the project, and it is a useful way of thinking when estimation needs to occur.

This is a useful method for being time-efficient. It can give a quick and rough estimate for how much work is expected for a project. The sizes can be converted into numbers at a later stage – when the team assigns a relative size to the project on hand. This is decided through discussion and collaborative efforts to understand everything that needs to be done.

If estimators propose sizes that do not match up, then the team voices their opinions on the topic and must eventually reach a consensus.

This is a pretty informal method that is great to use for a large number of items. Unfortunately, story points can be tricky. What might seem one size to one person could be perceived as another

by someone else. Whilst this can create some confusion, this method is based on open discussion, and everyone should get the chance to have their say.



Dot Voting

Sometimes it can be hard to order the items in the product backlog. This ranking method enables you to sort these items from highest to lowest priority, so you know where to focus your efforts. To do this, you need to select the most important user stories.

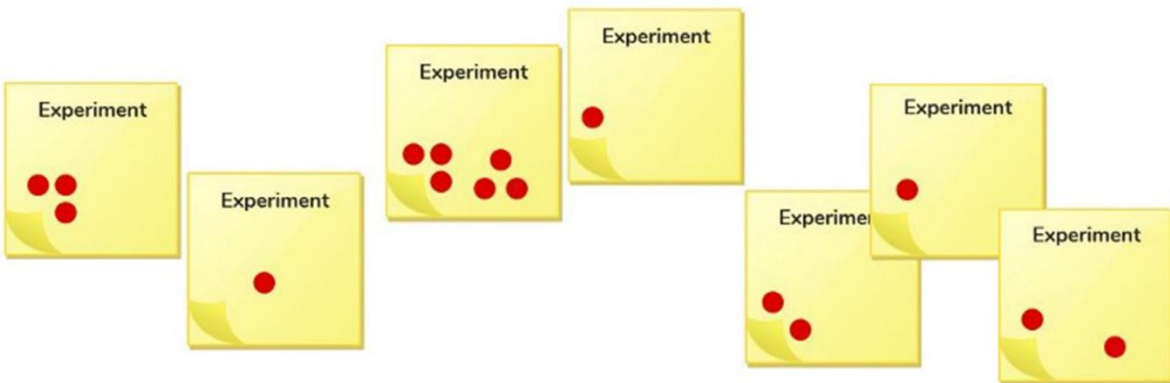
Start by posting all of the stories you need to deal with on a wall somewhere (or a board, if you're feeling more conventional). The posts should contain the story description and should look unique so that they are easily distinguishable to voters.

Team members involved in the process are all given 4 to 5 dots to dole out. The use of stickers or markers can be effective for this process. Team members place these dots on the user stories that they would prefer to start working on and distribute them throughout the options.

A leader then orders the stories from most preferred to least preferred (most preferred would be the story with the most marks or dots on it). If there is any stakeholder who is not happy with the order, then the items are divided into three groups – high, middle and low priority. Team members vote on the high priority stories again until they reach a consensus.

This is not necessarily an agile estimation technique – it's more of a decision-making tool. But if you have a small number of items – it can be very efficient. It's also really simple and is a good visualization tool.

“Let’s Dot Vote!”



The Bucket System

The Bucket System estimation technique is a relative estimation technique that best suits when there is a need to estimate a large number of product backlog items. The Bucket System can incorporate a large team in the estimation process and ensure fast and accurate estimates. In the Bucket System technique, the bucket represents numbered sticky notes that are placed on the wall or table in a sequential manner. The bucket numbers mostly range from 0-200, such as 0, 1, 2, 3, 4, 5, 8, 13, 20, 30, 50, 100, and 200. The team discusses the items to be estimated, places them under those numbered sticky notes (buckets), and reaches the final team's estimate. Overall, Bucket System presents a simple and fast way of estimating large stories count. Usually, when the stories count is greater than 40 but less than 500, the Bucket Estimation technique seems best to estimate effectively and efficiently.



Steps Involved in the Bucket System Estimation Technique

For newcomers in the agile-based development, The Bucket System estimation technique presents as the simplest estimation technique they can understand and deploy easily. Following are the steps agile teams have to follow to execute the Bucket System estimation technique rightly:

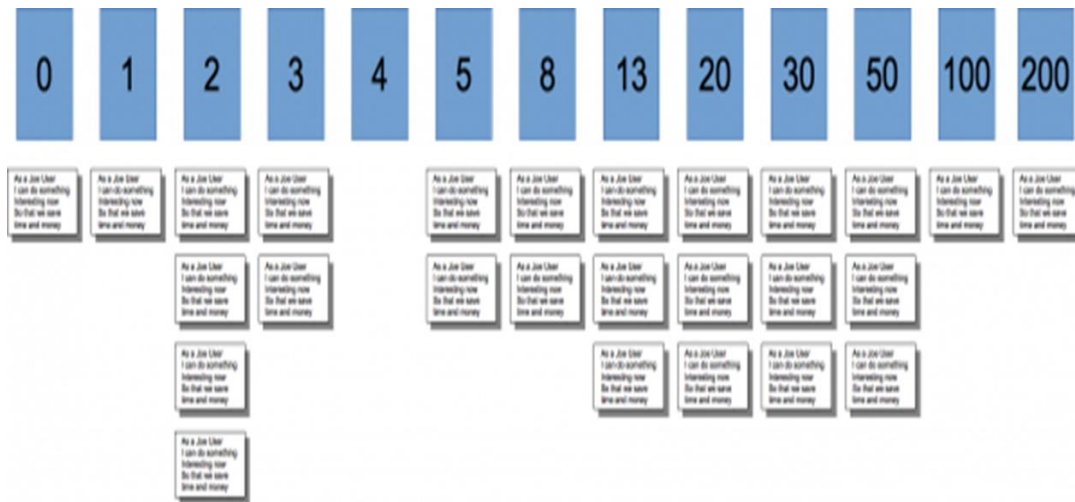
1. Arrange a team meeting and gather all the members in an on-premises meeting room. Arrange numbered sticky notes on the table or whiteboard. You can number them as 0, 1, 2, 3, 4, 5, 8, 13, 20, 30, 50, 100, and 200 or in other numbering formats as you like. The preferred format is to use the Fibonacci sequence. Moreover, the meeting facilitator should have written all the items to be estimated on the card or piece of paper so that they can be placed under the buckets.
2. Pick one item randomly, read it to the team, and then put it under the bucket that is labeled "8". This story will now be used as the reference for all other stories. For example, if the next story is less difficult than this one, then it will be placed under the bucket whose value is lower than 8.
3. Pick another item randomly and read it to the team including all its requirements and features. The team finalizes its position in the bucket compared to the item placed in Bucket 8. Afterward, the item is placed in the agreed bucket.
4. Pick another item randomly, discuss it, reach a consensus, and place the item in the appropriate bucket.
5. If the random selection of items has skewed the scale to one end, then the team should re-scale the items. For example, if the team realizes that the first item is very small, then it should be placed in the "1" bucket.
6. Divide the remaining items equally among all the members participating in the estimation. Every member reads the item on their own and places it under the appropriate bucket without discussing it with others. In case a member finds an item that is difficult to understand, then he/she can hand it over to another member.
7. Once all the items are placed under buckets, then all members quietly review the items and ensure that they are placed rightly. If someone finds that an item is not placed in the right bucket, then the team can quickly discuss it and reach a consensus.
8. Once the estimates of all items are final, write the bucket numbers on the items cards to record the estimate.

What is the Affinity Estimation Technique?

Affinity Estimation is a method of relative estimation of a large number of user stories in story points. In this estimation technique, user stories are grouped based on their relative size, i.e., size in relation to one another.

To understand it better, consider that you have three items, namely E, F, and G. The item G is the largest, item E is the smallest, and item F is between E and G. So if you have to estimate them, you can first take item F and place it on the scale. Next, you will take item G and see if it is larger than F or not. Since G is larger than F, so you will place it on the right side. Similarly, you can compare item E with F and then place E on the left side. This is exactly what you do in the Affinity Estimation technique.

In Affinity Estimation technique, the team quickly performs relative sizing of user stories by first taking a few cards and then moving them until every card is arranged on their relative size, i.e., smallest to largest.



Steps Involved in the Affinity Estimation Technique

The Affinity Estimation technique is best-suited when the team is small and wants to quickly do estimates of a large number of user stories. The steps involved in the Affinity Estimation technique can be divided into three main steps, as follows:

Step 1. Individual Relative Sizing of User Stories

1. Take a horizontal scale and mark its left end with "Smallest" and its right end with "Largest".

2. The product owner distributes the user stories among the team members usually in the form of cards.
3. Team members quietly read, understand, and estimate their stories without any discussion and then place the items in the correct position on the scale, which could be any position between the two ends of the scale.

Step 2. Collaborative Editing of the Scale

When all members have estimated their stories and the scale now contains all the stories, the members can do collaborative editing of the scale. So, the team members start discussing the stories with one another, including the development steps and challenges associated with the stories. Members can even clarify concerns directly with the product owner.

Once the members are done with the discussion, they will re-arrange the scale for those stories they believe are placed wrongly

Step 3. Placing User Stories in Buckets

1. The team now has to assign story points to each user story. To do that, the scale from smallest to largest is now divided and marked with XS, S, M, L, and XL (if the team is using the T-Shirt Sizing approach) or mark with other values depending on the team's decision.
2. The members now place the estimated user stories beneath the right mark (bucket). Once done, the team can assign story points to the user stories by following the below point scale:
 - XS - 1 point
 - S - 2 point
 - M - 3 point
 - L - 5 point
 - XL - 8 point

That's it. This way, the team can quickly and easily estimate a large number of user stories without going into deep discussions and consuming a lot of time.

Key Benefits of the Affinity Estimation Technique

Some of the key benefits of the Affinity Estimation technique are as follows:

- It is a simple and easy technique that does not require any technical knowledge.

- It makes estimation of a large number of backlog items quick.
- It encourages collaborative estimation, thereby resulting in a better understanding of the backlog items.
- It assists new agile teams in seamlessly estimating product backlog items.

Overall, the Affinity Estimation technique is best if teams want to quickly get over with estimation of a large number of backlog items without stressing on in-depth details of each item.

What is an Agile release plan?

What is a release plan? An Agile release plan is a [dynamic document](#) that breaks down how and when the organization releases product features or functionalities. This plan prioritizes feedback from previous iterations and sets out the scope, timeline, and resources for each release.

Agile release plans help teams decide how much functionality can be provided and how long it will take to develop. You can think of it as a prioritized list of functionalities to be delivered to the market, for an improved customer experience, over time.

In large, multi-departmental companies, release plans are excellent tools for communicating product status and progress with [cross-functional teams](#), leaders, and stakeholders involved in a [project](#), ensuring clear, aligned expectations for product development.

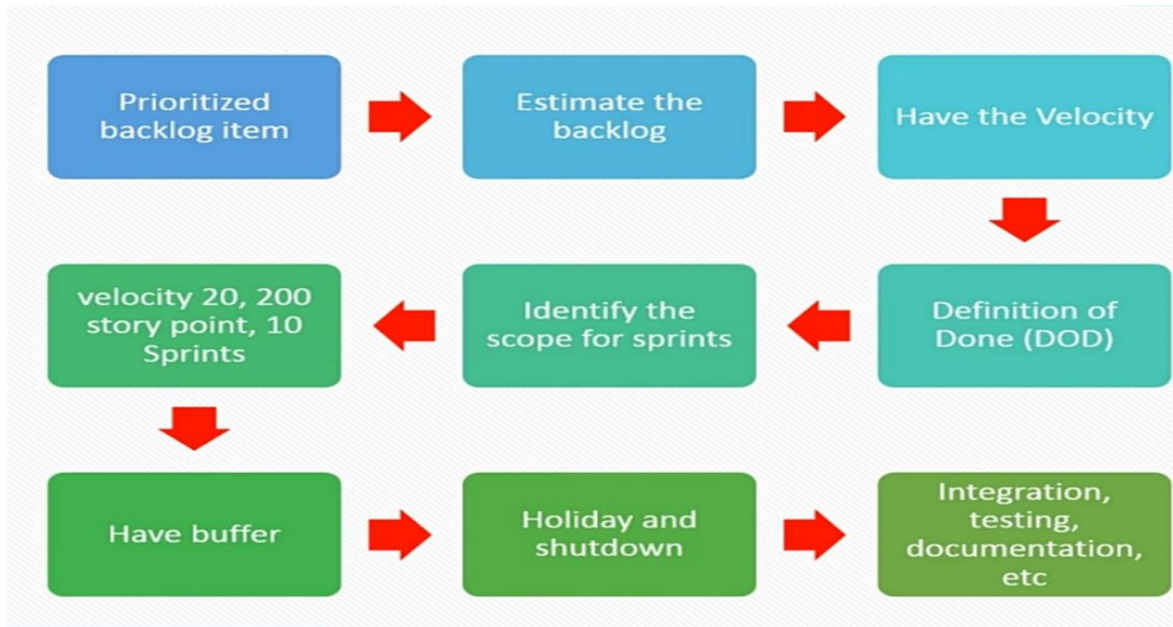
What is the purpose of release planning in Agile and Scrum?

The purpose of release planning in Agile and [Scrum](#) is to ensure your project moves in the right direction, aligns with the overall product roadmap, and beneficially incorporates feedback.

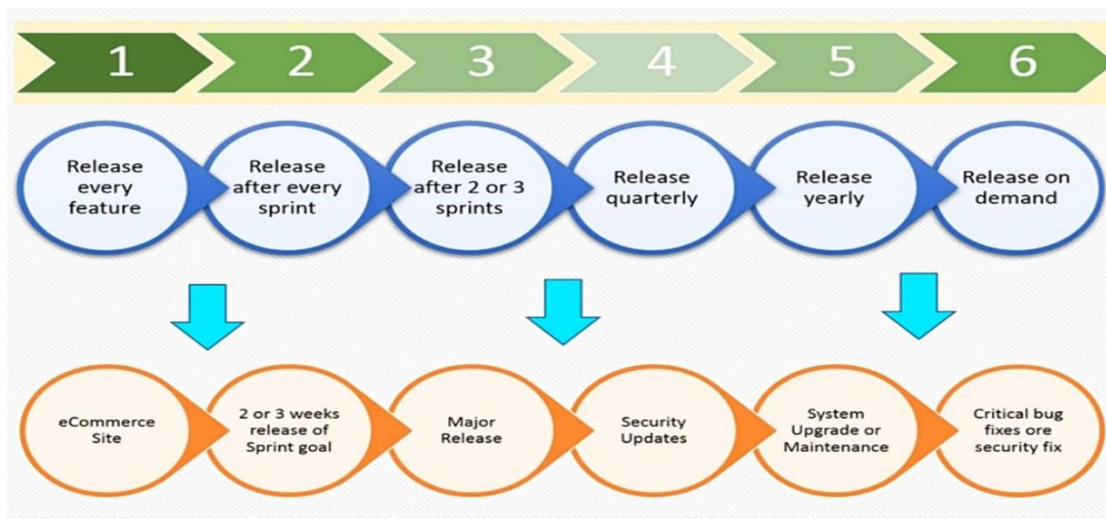
Agile and Scrum release planning are excellent for managing projects that require rapid development and testing within short periods. They provide an outward perspective for the development team, putting individual sprints in the context of the larger strategic product vision.

[Agile](#) and [Scrum](#) release planning are also great tools for combining [similar requirements](#) for improved customer experience. Commonly used in IT projects, a software release plan is a standard method of managing software development projects.

Agile releasing planning process



Release Types in Agile Project



What are agile prioritization techniques?

Prioritization is the process by which a set of tasks are aligned in order of importance. In [agile](#) development, establishing priorities helps employees accomplish work faster to deliver maximum business results on time.

The main purpose of agile prioritization techniques is to allocate resources to the projects that matter. Because agile product development can be more complex than regular development processes, prioritization techniques can make entire projects run more smoothly. Additionally, agile techniques involve four levels of priority—urgent, high, medium, and low—which force teams to take things one important task at a time!

Agile prioritization techniques help teams determine what tasks should be worked on first so the group can deliver a high-quality product with what limited resources they have at their disposal.

Why is the agile prioritization technique important?

Each of us must decide what to prioritize in a variety of situations. At work, prioritization can help you tackle new challenges efficiently. Prioritization is important in agile development because it encourages you and your teammates to focus attention on a project's main tasks.

Implementing agile prioritization techniques forces teams to move fast and innovate as they go. This encourages effective communication among colleagues, prompt feedback, and continuous testing to reduce risk, thus improving the overall quality of work.

4 levels of prioritization techniques in agile development

- Low
- Medium
- High
- Urgen

Low

Low-priority tasks include issues like information requests and other “nice-to-haves.” These are the features that may make the final results polished, but they aren't essential to the project's overall success or rollout. For example, decorations are an obvious “nice-to-have” for a party, but they aren't essential to celebrate a birthday or entertain partygoers.

Medium

Medium-priority tasks are things you'd like to get done but could live without at the end of the day. These tasks are non-critical to the usability of the final product. Let's use the party example again. While it might make the party better, having a DJ isn't something that will make or break your gathering, so it could be considered a medium-priority task.

High

High-priority tasks include things that aren't absolutely critical but that you want to complete more than others. At a party, guests aren't critical, but you probably really want to invite them. Therefore, delivering invitations could be considered a high-priority task.

Urgent

Urgent tasks are the number one priority. These tasks aren't up for debate because they may cause substantial loss, corruption of the system, customer issues, or total product failure if you don't complete them. When planning a party, the most urgent task might be as simple as setting an alarm that reminds you that the party is happening!

agile prioritization techniques

A DEEP Product Backlog

The product backlog is characterized by four qualities, i.e., the PBIs in the PB are detailed appropriately, estimated, emergent in nature, and prioritized. The acronym for it is DEEP.

- Detailed appropriately: It means the high priority items in the product backlog are listed in more detail (or fine-grained) as compared to the low priority items, which are coarse-grained.
- Estimated: The PBIs are estimated. The high priority items can be estimated in story points, ideal days, or any other estimate, whereas the low priority items can be in the form of epics.
- Emergent: The product backlog is not a static artifact. When new items are discovered, they are added to the bottom of the product backlog.
- Prioritized: The items in the product backlog are prioritized, with the highest priority PBIs on top of the product backlog (these items are pulled to be executed in the upcoming iteration). The prioritization of the items is generally done with respect to the value delivered, though there can be other factors as well. We will see more on this shortly.



Kano model

The Kano Model is a customer satisfaction-based prioritization framework. Researcher Noraki Kano developed the model to help product managers prioritize features and updates based on customer needs.

It has three parts:

- **Basic features:** Customers expect these essential functions in your product's performance, such as the ability to share a post on a social network.
- **Performance features:** These features increase customer satisfaction and make your product more enjoyable to use, like faster load times.
- **Delighters:** Unexpected features make customers happy. Examples of delighters include whimsical in-app messaging or the ability to use GIFs in posts.

MoSCoW method

The MosCow Method is a four-step process for prioritizing product requirements around their return on investment (ROI). It stands for “must haves,” “should haves,” “could haves,” and “will not haves.” Apply this method company-wide to help frame discussions around what will increase revenue.

Here's a breakdown of the model:

- **Must Have (M):** These are the requirements needed for the project's success.
- **Should Have (S):** These are important requirements for the project but not necessary.
- **Could Have (C):** These requirements are “nice to have.” But don't have as much impact as the others.
- **Will Not Have (W):** These requirements aren't a priority for the project.

Practice Question :

1. Demonstrate value-driven development with suitable example.(April 2022)
2. Explain the roles of scrum master, product owner and development team. .(April 2022)
3. Write short note i) Role of project manager .(April 2022)
4. What are the various techniques to estimate story points in agile?(Oct.2022)
5. “Splendid Holidays”, one of the distinguished service providers of International and Domestic Tour & Travel. Due to their reliable and flexible services, they have earned a

large number of customers across the nation. They want to upgrade their ERP software. As a product owner you have to identify all possible epic, Features and User Stories for the same. (April 2023)

6. Explain DSDM
7. Write Short Note On;- I) Product Vision II) Product Strategy III) Product RoadMap IV)Project Objectives V)Agile Release Plan VI) Product Prioritization